

Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial
15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics
Master of Data Science



Thesis Title

THESIS to obtain the **DEGREE** of
MASTER OF DATA SCIENCE

A thesis presented by:
Joey Ramone

Thesis Advisors:
Dr. Willie E. Coyote
Dr. Homer J. Simpson

Tlaquepaque, Jalisco, January, 2021

Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial 15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics Master of Data Science Approval Form

Thesis Title: **Thesis Title**

Author: **Joey Ramone**

Thesis Approved to complete all degree requirements for the Master of Science Degree in Data Science.

Thesis Advisor, **Dr. Willie E. Coyote**

Thesis Co-Advisor, **Dr. Homer J. Simpson**

Thesis Reader, **Dr. Frederick Flintstone**

Thesis Reader, **Dr. Farrokh Bulsara**

Academic Advisor, **Dr. Maryam Mirzajani**

Tlaquepaque, Jalisco, January, 2021

Thesis Title

Joey Ramone

Abstract

Here

Thesis Title

Joey Ramone

Resumen

En esta página debes escribir el abstract pero en español.

Contents

	Page
1 Introduction	19
1.1 Learning from Graph-Structured Data	19
1.2 Objectives and Scope	20
1.3 Structure of the Document	22
2 Learning Tasks on Graphs	23
2.1 Node-Level Tasks	24
2.2 Edge-Level Tasks	25
2.3 Graph-Level Tasks	26
3 Fundamentals of Graph Theory	27
3.1 Formal Definitions	27
3.2 The Adjacency Matrix and Graph Types	28
3.3 Walks, Connectivity, Homophily, and Isomorphism	31
3.4 The Graph Laplacian and the Spectral Perspective	32
3.5 Notation Summary	34
4 Neural Networks and Their Limitations on Relational Data	35
4.1 The Multilayer Perceptron	35
4.2 Training by Gradient Descent	36
4.3 Why the Multilayer Perceptron Fails on Graphs	38
5 Inductive Bias, Invariance, and Equivariance.	41
5.1 Inductive Bias in Neural Architectures	41
5.2 The Relational Inductive Bias	42
5.3 Invariance and Equivariance	43
5.4 Permutation Invariance and Equivariance on Graphs	44
Bibliography	47

List of Figures

	Page
1.1 Examples of graph-structured data.	20
2.1 The three levels of graph learning tasks.	23
2.2 Node-level tasks.	24
2.3 Edge-level tasks.	25
2.4 Graph-level tasks.	26
3.1 A graph and its adjacency matrices under two node indexings.	29
3.2 A directed graph and a weighted graph.	30
3.3 Self-loops, a multigraph, and a bipartite graph.	30
3.4 A homogeneous and a heterogeneous graph.	31
3.5 A walk and a shortest path.	31
3.6 A disconnected graph and its components.	32
3.7 Two isomorphic graphs.	32
4.1 The multilayer perceptron.	36
4.2 Neighborhoods of varying size.	38
4.3 The same graph under two node orderings.	39
5.1 Permutation invariance and equivariance on a graph. . .	45

List of Tables

	Page
3.1 Notation introduced in this chapter.	34

Dedicated to:

Dedicado a:

Escribir la dedicatoria pero en español.

1 Introduction

Contents

1.1	Learning from Graph-Structured Data	19
1.2	Objectives and Scope	20
1.3	Structure of the Document	22

1.1 Learning from Graph-Structured Data

Data in machine learning often comes in a few familiar forms. Most datasets are tabular: a table of rows and columns in which each row is an independent example described by the same fixed set of features. Other data is organized as images, that is, grids of pixels, or as sequences such as text and audio, whose elements follow one another in a specific order. What these forms share is regularity: each has a fixed, well-defined structure that repeats across every example.

Many real-world systems do not fit any of these shapes. A molecule is a set of atoms connected by chemical bonds, and neither the number of atoms nor the way they are connected is the same from one molecule to the next. A social network is a set of people connected by relationships, where each person may be linked to a different number of others. A citation network is a set of papers that reference one another, each citing a different set of others. These are not tables, images, or sequences; they are networks of entities and the relations between them (Figure 1.1). Data of this kind is naturally described by a *graph*: a set of entities, called *nodes*, together with a set of connections between them, called *edges*. Both nodes and edges can carry their own data, for example in a molecule, each node is an atom labeled with its element (carbon, hydrogen, oxygen, and so on) and each edge is a bond of a particular type (single, double, and so on); in a social network, the nodes are people and the edges are the relationships among them.

Rather than treating a domain as a collection of isolated data points, we describe it in terms of entities and the relationships among them, which means there is an underlying graph whose structure ties those

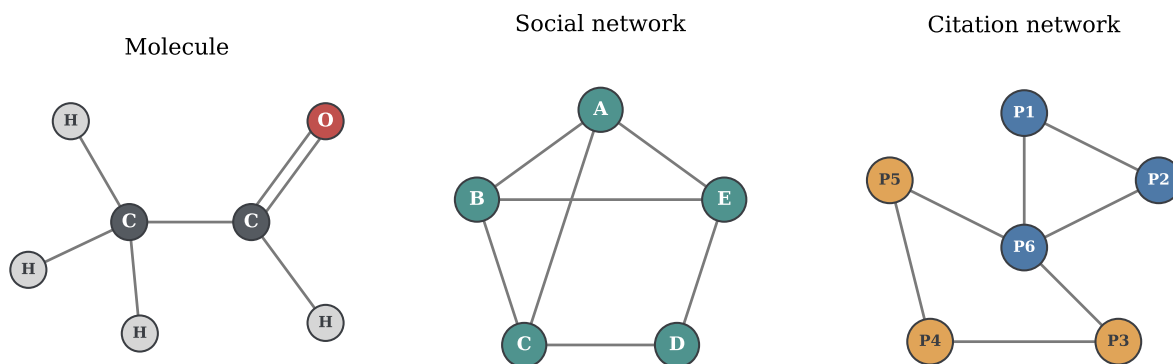


Figure 1.1: Three domains represented as graphs. A molecule, whose nodes are atoms labeled by their element and whose edges are bonds (single or double); a social network, whose nodes are people and whose edges are relationships; and a citation network, whose nodes are papers (colored by topic) and whose edges are citations between them. In each case the entities and their connections, not a table, grid, or sequence, define the data.

entities together. Seen this way, the graph is not merely one possible encoding of a molecule or a social network but a faithful representation: it records exactly which entities exist and how they relate, without forcing the data into a shape it does not have. A wide range of domains exhibit this kind of relational structure, and modeling them explicitly as graphs, rather than ignoring the connections, can lead to more accurate predictions. Data organized in this way is called *graph-structured data*.

Most deep learning models are tailored to the regular structures described above, particularly sequences and images laid out on a grid. Graphs are harder to work with. They have no fixed size, since different graphs contain different numbers of nodes. They have no fixed layout: a pixel in a grid has a clear sense of place, with neighbors above, below, and to either side, whereas a node in a graph has no such spatial arrangement and may be connected to any number of others. And they have no natural ordering, with no first or last node and no canonical point from which to read the rest.

The goal, then, is a model that takes a graph as input and produces predictions about it, such as a label for each node, whether connections exist between pairs of nodes, or a single label for the graph as a whole. This raises the central question: how can a predictive model – concretely, a neural network – be designed to operate directly on a graph, respecting its variable size and its lack of order instead of discarding them? Neural networks built for this purpose are called *Graph Neural Networks*, hereafter abbreviated as GNNs.

1.2 Objectives and Scope

The general objective of this work is to provide a self-contained reference on GNNs for homogeneous graphs under the *message-passing* framework: a single document that develops the theory that motivates

and defines these models, presents representative architectures, and evaluates them through reproducible implementations.

Specifically, the work aims to:

- establish the foundations that motivate GNNs: how graphs represent relational data, why standard neural networks cannot learn from it directly, and the inductive biases a model on graphs must respect;
- develop the message-passing framework as the unifying formulation from which GNN architectures are derived;
- present representative architectures for the three standard learning tasks on graphs: the Graph Convolutional Network (GCN) for node classification, the Graph Auto-Encoder (GAE) for link prediction, and the Graph Isomorphism Network (GIN) for graph classification;
- implement the three architectures on standard benchmark datasets (Cora, Amazon Computers, and NCI1) using PyTorch Geometric, with reproducible implementations, and evaluate each against a multilayer perceptron baseline that ignores the graph, examining the predictive benefit of explicitly modeling the relational structure of the data;
- examine the principal limitations of GNNs under this framework, including *oversmoothing* and the ceiling on their expressive power.

Its contribution is the reference document itself: a single, reproducible source that develops the message-passing framework from first principles, presents representative GNN architectures, and serves as a foundational resource for studying the field.

The graphs we consider are treated mainly as *undirected*, meaning their connections carry no direction, and *simple*, meaning that at most one connection joins any pair of nodes and that no node is connected to itself. This choice reflects simplicity rather than a limitation of the framework: message passing has enough design space to accommodate *directed* graphs, *self-loops* (a node connected to itself), and even *multigraphs* (more than one connection between the same pair of nodes).

The graphs considered in this work are also *homogeneous*: they contain a single kind of node and a single kind of connection. This restriction is deliberate. The homogeneous setting is where the message-passing framework is most directly understood, and it is the foundation on which more elaborate models are built, including *heterogeneous* graphs, which carry several kinds of nodes and connections and require machinery beyond the scope of this work. Establishing the homogeneous case is a prerequisite for these extensions, which we

leave as future research. The formal definitions of these graph types are given later in the document.

1.3 *Structure of the Document*

This introduction has set out the motivation, objectives, and scope of the work. The remainder of the document is organized as follows.

The first chapters establish the foundations. Chapter 2 surveys the tasks that graph learning addresses at the node, edge, and graph level, together with the practical applications that motivate the rest of the work. Chapter 3 establishes the formal definitions and notation for graphs used throughout. Chapter 4 reviews the multilayer perceptron and shows why standard neural networks are ill-suited to relational data. Chapter 5 develops the notion of inductive bias and the permutation invariance and equivariance that any graph layer must satisfy.

Chapter ?? builds on these foundations to derive the message-passing framework: the message, aggregation, and update functions, the receptive field that successive layers induce, the expressive power of the resulting models, and the architectural design space it enables.

Chapter ?? presents the three representative architectures within this framework: GCN for node classification, GAE for link prediction, and GIN for graph classification. Chapter ?? turns to practice, providing reproducible implementations of these architectures on the Cora, Amazon Computers, and NCI1 benchmarks in PyTorch Geometric, with the accompanying code available in a companion repository.

Chapter ?? examines the limitations of GNNs under the message-passing framework, and Chapter ?? synthesizes the work and outlines directions for future research.

2 Learning Tasks on Graphs

Contents

2.1	Node-Level Tasks	24
2.2	Edge-Level Tasks	25
2.3	Graph-Level Tasks	26

Once a domain has been represented as graph-structured data, a natural question arises: what machine learning tasks can be defined on a graph? A graph can be viewed at three levels of granularity: its individual nodes, its edges, and the graph as a whole. Each level gives rise to a family of prediction tasks: predicting a property of each node, predicting the existence or type of a connection between pairs of nodes, or predicting a property of the entire graph. These are known, respectively, as *node-level*, *edge-level*, and *graph-level tasks* (Figure 2.1), and they structure the architectures and implementations developed in the rest of this work.

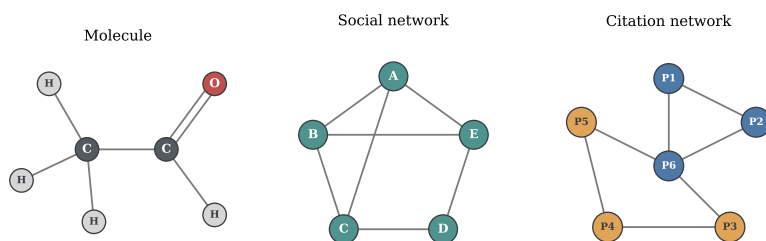
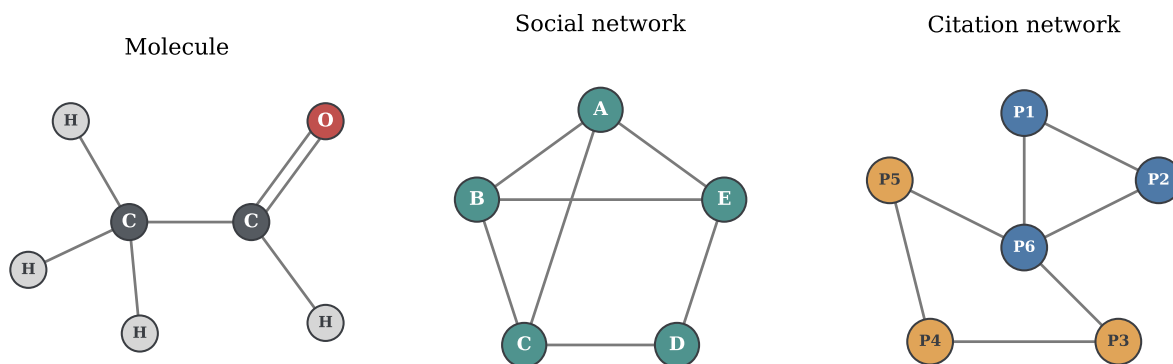


Figure 2.1: The three levels at which a prediction can be made on a single graph. A node-level task predicts a property of an individual node (highlighted); an edge-level task predicts a property of a connection between two nodes, such as whether it exists (dashed); and a graph-level task predicts a single property of the entire graph (framed). The same object supports all three.

These three levels correspond to the standard categories of graph learning tasks, but they do not exhaust the field. Other settings include *community/subgraph-level tasks*, which focus on groups of related nodes, and *graph generative tasks*, which aim to generate new graphs, such as candidate molecular structures with desirable properties. Although these settings rely on the same representational principles introduced here, they lie outside the scope of this work, which focuses on the three standard task levels.



The following sections examine each of these levels in turn and present representative applications that motivate them.

2.1 Node-Level Tasks

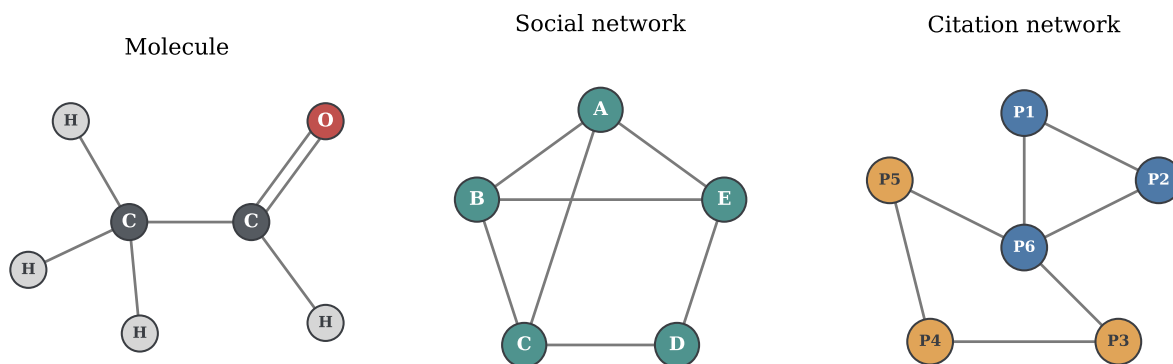
A *node-level task* asks for a prediction about each individual node of a graph, drawing on both the node's own attributes and its position within the surrounding structure. The most common instance is *node classification*: assigning each node to one of a set of categories. A representative setting is a citation network, in which the nodes are scientific papers, the edges are citations between them, and the goal is to infer the topic of each paper (Figure 2.2). A paper's own text is informative, but so is its neighborhood: papers tend to cite others on related subjects, so the topics of a paper's neighbors are strong evidence for its own. Node classification exploits exactly this, combining what a node contains with what surrounds it. The same pattern appears whenever entities in a network must be categorized.

The target need not be a category. A graph representation of a protein, for example, can be constructed where nodes represent amino acid residues and edges connect residues that are close in space. One may predict a discrete property of each residue, such as its functional role, or a continuous property, such as its position in three dimensions, a formulation that underlies recent advances in protein structure prediction¹. A node-level task can therefore be classification or regression, depending on whether the per-node target is categorical or numerical.

The same framing extends to economic settings. In a financial network whose nodes are accounts and whose edges are transactions between them, fraud detection amounts to flagging each account as legitimate or fraudulent: a node classification problem in which

Figure 2.2: Left: the node classification task in the abstract. Some nodes carry a known label (shown by color) while others are unknown (marked with a question mark); the model predicts a label for every node from its attributes and the surrounding structure. Right: a concrete instance as a citation network, where each node is a scientific paper and the predicted label is its topic.

¹ John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873): 583–589, 2021. DOI: 10.1038/s41586-021-03819-2



fraudulent behavior is often more evident in the surrounding network of connections than in any single account considered in isolation.

2.2 Edge-Level Tasks

An *edge-level task* focuses on the relationships between entities rather than the entities themselves. The central example is *link prediction*: given the edges already present in a graph, predict which further edges are likely to exist. Recommendation is one of the most familiar application (Figure 2.3). In its simplest form, a network records products that are frequently bought together, with the nodes being products and edges indicating co-purchases. Predicting missing edges in this network amounts to identifying product pairs that are likely to be associated, even if they have not yet been observed together. A richer version introduces two kinds of entity, users and products, with an edge representing interactions such as purchases or ratings; here, link prediction becomes the recommendation problem itself, anticipating which products a user will want next.

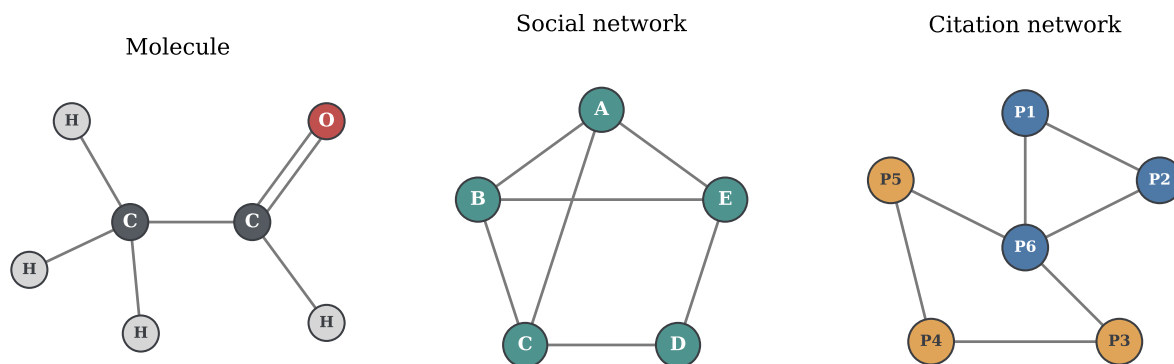
What makes the graph formulation effective is that it learns from the relational structure rather than treating each entity in isolation. A recommender system that treats products only by their individual attributes, ignoring the network of interactions around them, discards precisely the signal that reveals which items behave similarly; methods that propagate information along the graph structure consistently outperform such approaches, a principle that underlies graph-based recommender systems deployed at web scale ².

Link prediction extends well beyond commerce. Given a pair of drugs, one may predict whether taking them together produces an adverse interaction, by placing both within a larger biological network and asking whether a particular kind of connection should join them ³.

Figure 2.3: Left: the link prediction task in the abstract. Solid edges are observed connections; dashed edges marked with a question mark are candidate connections whose existence the model must predict. Right: a concrete instance as a recommendation setting, where the two kinds of entity (users, drawn as squares, and products, drawn as circles) are joined by observed interactions, and the task is to predict further user-product links.

² Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890

³ Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294



The same formulation is also used to complete *knowledge graphs*, large networks of facts in which entities are connected by relations: many true relations are simply unobserved, and link prediction proposes the most plausible ones.

2.3 Graph-Level Tasks

A *graph-level task* treats an entire graph as a single example and predicts a property of the whole. The representative task is *graph classification*: assigning a label to a complete graph (Figure 2.4). Here the data is not one large network but a collection of many separate graphs, each with its own label, and the model must summarize the structure and attributes of an entire graph into a single prediction. Molecular property prediction is the canonical example. A molecule is naturally a graph, with atoms as nodes and chemical bonds as edges, and many prediction tasks about it are graph-level: whether a compound is toxic, or whether it is active against a given biological target. Because molecules vary in size and structure, this is exactly the regime that calls for a model able to handle graphs of varying structure and return one prediction per graph.

This capability feeds directly into drug discovery, where large libraries of candidate compounds are screened to prioritize the few worth synthesizing and testing in the laboratory, a process in which an accurate graph-level predictor can save considerable experimental effort. The same idea applies in materials science, where the atomic structure of a material is represented as a graph and a graph-level model predicts a physical property such as its stability or its mechanical strength.

Across all three levels, the graph is used in different ways, with the level of the task determining what is predicted and how a model must process the structure to predict it.

Figure 2.4: Left: the graph classification task in the abstract. The data is a collection of distinct graphs, differing in size and shape, each assigned a single label for the graph as a whole. Right: a concrete instance as molecular property prediction, where each graph is a molecule (atoms as nodes, bonds as edges) labeled by a property such as being active or inactive against a target.

3 Fundamentals of Graph Theory

Contents

3.1	Formal Definitions	27
3.2	The Adjacency Matrix and Graph Types	28
3.3	Walks, Connectivity, Homophily, and Isomorphism	31
3.4	The Graph Laplacian and the Spectral Perspective	32
3.5	Notation Summary	34

The applications surveyed in Chapter 2 share a single underlying object: the graph. To move from describing tasks to formulating them precisely, that object must itself be defined formally. This chapter introduces the formal language of graphs used throughout the rest of this work. It first defines graphs, neighborhoods, and node and edge features; it then encodes graph structure algebraically through the adjacency matrix and distinguishes the main types of graphs; it examines the structural properties, from walks to isomorphism, on which later developments rely; it introduces the graph Laplacian and the spectral perspective it provides; and it closes with a summary of the notation fixed here.

3.1 Formal Definitions

A *graph* is a pair

$$\mathcal{G} = (V, E), \tag{3.1}$$

where V is a finite set of *nodes*, also called vertices, and E is a set of *edges*, also called links, each one an unordered pair of nodes recording which two nodes are connected ¹. The number of nodes is denoted $n = |V|$ and the number of edges $m = |E|$. In classical graph-theoretic notation, generic nodes are denoted by $u, v \in V$, and an edge between them by the unordered pair $\{u, v\} \in E$; at this stage, the notation simply states that the two nodes are connected.

Because V and E are sets in the mathematical sense, neither carries an intrinsic order: a graph specifies which nodes are connected, and

¹ Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5 edition, 2017. ISBN 978-3-662-53622-3

nothing more. It is nevertheless convenient, particularly when graphs are represented by matrices, to fix an indexing of the nodes by integers. Throughout this work, nodes are therefore denoted $i, j \in \{1, \dots, n\}$, with the convention that i refers to the node under consideration and j to a neighboring node whenever a neighborhood relation is involved. The choice of indexing is arbitrary and carries no information about the graph itself, a point this chapter returns to.

Two nodes are *adjacent*, or *neighbors*, when an edge connects them. The *neighborhood* of a node i is the set of its neighbors,

$$\mathcal{N}(i) = \{j \in V : \{i, j\} \in E\}, \quad (3.2)$$

and the *degree* of node i is the number of neighbors it has,

$$\deg(i) = |\mathcal{N}(i)|. \quad (3.3)$$

In a social network, for example, the neighborhood of a person is the set of people connected to them, and the degree counts them.

Beyond its structure, a graph typically carries data ². Each node i has an associated *node feature vector* $\mathbf{x}_i \in \mathbb{R}^d$, where d is the node feature dimension, describing the attributes of the entity the node represents, such as the textual content of a paper in a citation network, or the chemical element of an atom in a molecule. Stacking the n node feature vectors as rows produces the *node feature matrix*

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \quad (3.4)$$

whose i -th row is the transposed feature vector of node i . Writing the features as a matrix presupposes that the node indexing is fixed as above. Edges can carry attributes as well: the *edge feature vector* $\mathbf{e}_{i,j} \in \mathbb{R}^c$, where c is the edge feature dimension, describes the connection between nodes i and j , such as the type of a chemical bond in a molecule.

² William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

3.2 The Adjacency Matrix and Graph Types

With the nodes indexed from 1 to n , the structure of a graph can be encoded in a single matrix. The *adjacency matrix* of $\mathcal{G}$ is the matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ with entries

$$A_{ij} = \begin{cases} 1 & \text{if } \{i, j\} \in E, \\ 0 & \text{otherwise,} \end{cases} \quad (3.5)$$

so that each entry records the presence or absence of a single edge.

The adjacency matrix depends on the node indexing. Reordering

There are $n!$ possible indexings of n nodes, and each yields a valid adjacency matrix of the same graph.

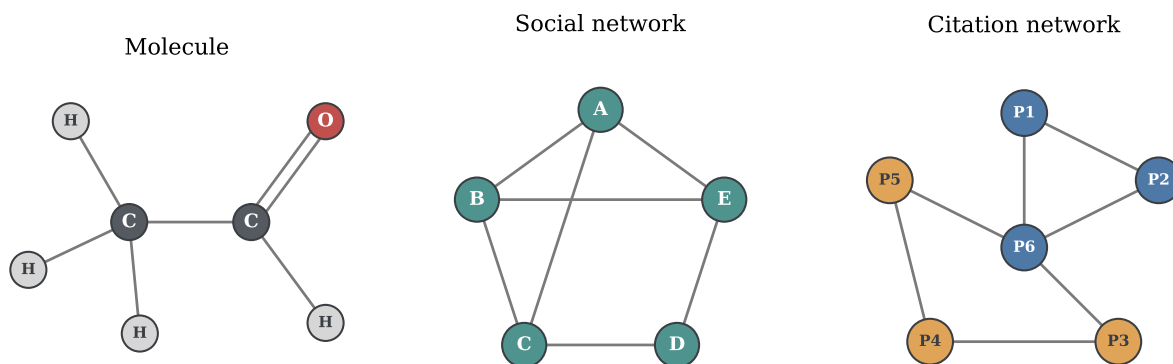


Figure 3.1: A graph on six nodes (left) and its adjacency matrices under two different indexings of the nodes (center and right). Both matrices describe the same structure: reordering the nodes permutes the rows and columns of $\mathbf{A}$.

the nodes permutes the rows and columns of $\mathbf{A}$, producing a different matrix that describes exactly the same graph. Figure 3.1 shows a graph together with its adjacency matrices under two different indexings: a graph has not one adjacency matrix but one per indexing, and no single one of them is privileged.

The first distinction among graph types concerns the orientation of edges. In an *undirected* graph, the setting assumed so far, an edge $\{i, j\}$ has no orientation and can be traversed in either direction; its adjacency matrix is consequently symmetric, $\mathbf{A} = \mathbf{A}^\top$. In a *directed* graph, edges are instead ordered pairs: the pair $(i, j) \in E$ denotes an edge from a *source* node i to a *target* node j , and the presence of (i, j) no longer implies the presence of (j, i) . The adjacency entry reads $A_{ij} = 1$ exactly when $(i, j) \in E$, so $\mathbf{A}$ is in general not symmetric and row i lists the targets of the edges leaving node i . The degree of a node accordingly splits in two: its *out-degree*, the number of edges leaving it, and its *in-degree*, the number of edges arriving at it. Unless stated otherwise, the graphs treated in this work are undirected.

A second distinction concerns whether edges carry a magnitude. In a *weighted* graph, each edge has an associated scalar weight $w_{ij} \in \mathbb{R}$, expressing, for instance, the strength of a relationship or a physical distance; the adjacency matrix then stores $A_{ij} = w_{ij}$ in place of 1 (Figure 3.2). In an *unweighted* graph, every edge counts the same and (3.5) applies as stated. A scalar weight can be viewed as the special case $c = 1$ of the edge feature vectors introduced in Section 3.1, that is, a single number attached to each edge.

A *self-loop* is an edge connecting a node to itself; in the adjacency matrix it appears as a nonzero diagonal entry, $A_{ii} \neq 0$. A *multigraph* allows several parallel edges between the same pair of nodes, a multiplicity that the binary entries of (3.5) cannot record. A graph with no self-loops and at most one edge between any pair of nodes

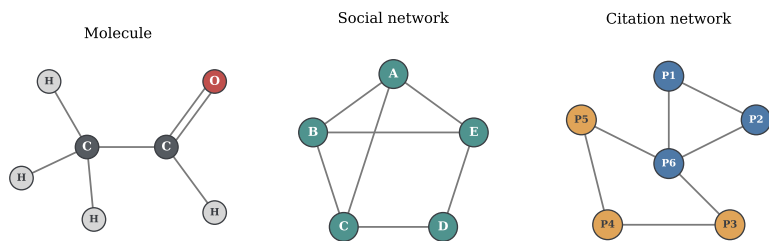


Figure 3.2: Left: a directed version of the graph of Figure 3.1 and its adjacency matrix. Edge orientations make $\mathbf{A}$ asymmetric: row i lists the targets of the edges leaving node i , and the in-degree and out-degree of one highlighted node are annotated. Right: a weighted graph, with edge thickness proportional to the weight w_{ij} .

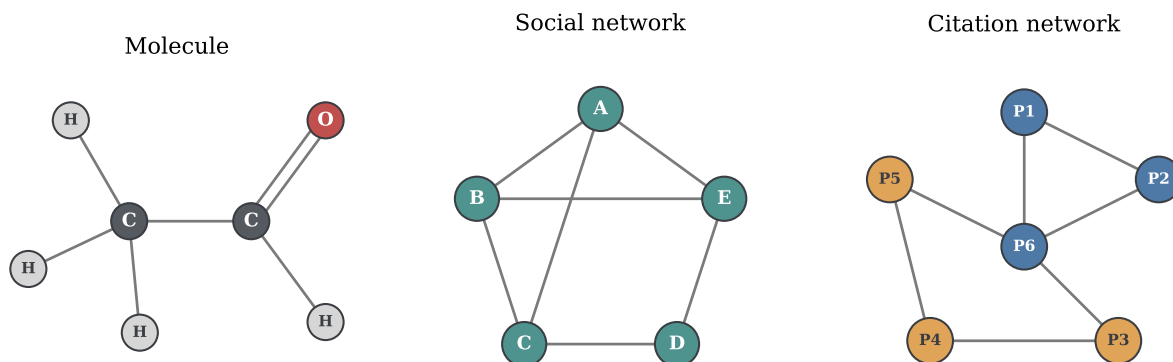


Figure 3.3: Left: a graph with self-loops. Center: a multigraph with parallel edges. Right: a bipartite graph, with its two subsets of nodes drawn as distinct shapes.

is called *simple*. A *bipartite* graph, in contrast, is not a departure from simplicity but a structural subclass of it: its nodes partition into two disjoint subsets, and every edge connects a node of one subset with a node of the other, as in a network of users and products joined by purchases. Because of this specificity, such graphs are explicitly named bipartite; a graph described only as simple implies no such partition. Figure 3.3 illustrates the three cases. As established in Chapter 1, the graphs treated in this work are simple.

The last distinction concerns the kinds of entities and relations a graph contains. A graph is *homogeneous* when all of its nodes represent one kind of entity and all of its edges represent one kind of relation: every node then shares the same feature space $\mathbb{R}^d$ and every edge the same feature space $\mathbb{R}^c$. A citation network is homogeneous, since every node is a paper and every edge a citation. A graph is *heterogeneous* when it contains several types of nodes or edges, each with its own semantics and, typically, its own feature space³. A citation network that includes both papers and their authors is heterogeneous: papers and authors are different kinds of entities, joined by a relation of authorship. Figure 3.4 contrasts the two settings. As stated in Chapter 1, this work treats homogeneous graphs.

The homogeneous/heterogeneous terminology comes from the network analysis and machine learning literature rather than from classical graph theory.

³ William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

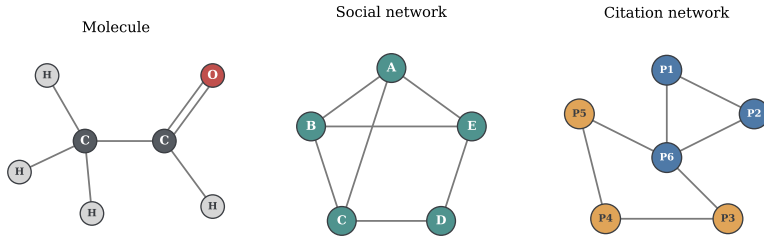


Figure 3.4: Left: a homogeneous graph, in which all nodes represent the same kind of entity and all edges the same relation. Right: a heterogeneous graph, in which node shapes denote different entity types and edge styles denote different relations.

3.3 Walks, Connectivity, Homophily, and Isomorphism

The definitions so far describe a graph locally: nodes, their neighbors, and their attributes. Several properties that matter in the remainder of this work are instead global, arising from how edges chain together across the graph. A *walk* is a sequence of nodes in which each consecutive pair is joined by an edge, and its *length* is the number of edges traversed; a walk may revisit nodes and edges. A *path* is a walk in which all nodes are distinct (Figure 3.5).

The *geodesic distance* between two nodes is the length of a shortest path between them,

$$d(i, j) = \min \left\{ k \in \mathbb{N} : \text{there is a path of length } k \text{ between } i \text{ and } j \right\} \quad (3.6)$$

with $d(i, i) = 0$. The distance is always written with its two arguments, $d(i, j)$, which distinguishes it from the node feature dimension d . When no path joins i and j , the distance is taken to be infinite, a situation characterized next.

A graph is *connected* when a path exists between every pair of its nodes, that is, when $d(i, j)$ is finite for all $i, j \in V$. When it is not, the graph decomposes into *connected components*: maximal groups of nodes that are mutually reachable, with no edges between groups (Figure 3.6). In directed graphs, connectivity refines into weak and strong variants according to whether edge orientations are respected; since this work treats undirected graphs, the refinement is not developed further.

Structure and attributes are often correlated. A graph exhibits *homophily* when adjacent nodes tend to have similar attributes, and *heterophily* when they tend to differ. The notion originates in the analysis of social networks, where similar individuals form ties at higher rates than dissimilar ones⁴, and it extends to many of the graphs of Chapter 2: in a citation network, a paper predominantly cites papers on related topics, so neighboring nodes tend to share a subject. Whether a graph is homophilous or heterophilous is an

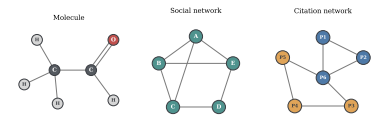


Figure 3.5: A walk that revisits a node (top) and a shortest path between the same endpoints (bottom), whose length is their geodesic distance.

⁴ Miller McPherson, Lynn Smith-Lovin, and James M. Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27(1):415–444, 2001. DOI: 10.1146/annurev.soc.27.1.415

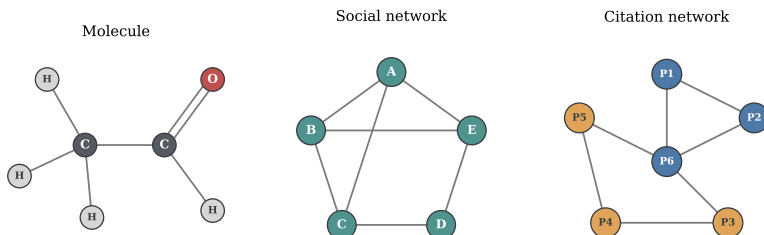


Figure 3.6: A disconnected graph with two connected components, one of which is a single isolated node.

empirical property of the data rather than a structural constraint, and the degree to which it holds varies from one graph to another.

The arbitrariness of node indexing observed in Section 3.2 raises a final question: when do two graphs that are presented differently have the same structure? Two graphs $\mathcal{G}_1 = (V_1, E_1)$ and $\mathcal{G}_2 = (V_2, E_2)$ are *isomorphic* when there exists a bijection $\pi : V_1 \rightarrow V_2$ that preserves adjacency,

$$\{i, j\} \in E_1 \iff \{\pi(i), \pi(j)\} \in E_2, \quad (3.7)$$

for all nodes $i, j \in V_1$. Isomorphic graphs are the same graph up to a relabeling of its nodes; the two adjacency matrices of Figure 3.1 are representations related in exactly this way. Two graphs drawn very differently may be isomorphic (Figure 3.7). Determining whether two graphs are isomorphic is a fundamental problem in graph theory. A classical heuristic, the Weisfeiler–Leman test, distinguishes most non-isomorphic graphs in practice ⁵, and it reappears later in this work in connection with the expressive power of graph neural networks.

3.4 The Graph Laplacian and the Spectral Perspective

The structural properties of Section 3.3 and the algebraic encoding of Section 3.2 meet in a single observation: powers of the adjacency matrix count walks. For any positive integer k , the entry in row i and column j of the k -th matrix power gives the number of walks of length k between the two nodes,

$$(\mathbf{A}^k)_{ij} = |\{\text{walks of length } k \text{ between } i \text{ and } j\}|, \quad (3.8)$$

a count that is symmetric in i and j for the undirected graphs treated here. In particular, the entry is nonzero exactly when the two nodes are joined by some walk of that length, which ties the reachability notions of Section 3.3 directly to matrix algebra and motivates the matrices introduced next.

The *degree matrix* $\mathbf{D} \in \mathbb{R}^{n \times n}$ is the diagonal matrix that holds the node degrees,

$$\mathbf{D} = \text{diag}(\deg(1), \dots, \deg(n)), \quad (3.9)$$

⁵ Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M. Kriege, Martin Grohe, Matthias Fey, and Karsten Borgwardt. Weisfeiler and Leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023. <http://jmlr.org/papers/v24/22-0240.html>

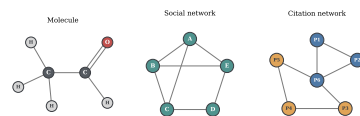


Figure 3.7: Two graphs that are drawn differently but are isomorphic: a bijection between their node sets preserves all adjacencies.

with every off-diagonal entry equal to zero. Combining it with the adjacency matrix yields the central operator of spectral graph theory ⁶, the *graph Laplacian*

$$\mathbf{L} = \mathbf{D} - \mathbf{A}. \quad (3.10)$$

Each diagonal entry of $\mathbf{L}$ is the degree of the corresponding node, and each off-diagonal entry is -1 where an edge is present and 0 otherwise. For the six-node graph of Figure 3.1, under the node indexing of its centre panel and with $\mathbf{D} = \text{diag}(2, 3, 2, 3, 3, 1)$, the Laplacian is

$$\mathbf{L} = \begin{bmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}.$$

The Laplacian admits a direct interpretation in terms of how values spread over a graph. Assigning a real number to each node defines a vector $\mathbf{f} \in \mathbb{R}^n$ called a *graph signal*, with component f_i the value at node i . The Laplacian measures how much such an assignment varies across edges through the quadratic form

$$\mathbf{f}^\top \mathbf{L} \mathbf{f} = \sum_{\{i,j\} \in E} (f_i - f_j)^2, \quad (3.11)$$

which sums the squared difference between the values at the two endpoints of every edge. The form vanishes when $\mathbf{f}$ is constant on each connected component and grows as the values assigned to neighboring nodes diverge. Because it is a sum of squares, the quadratic form is nonnegative for every $\mathbf{f}$, so the Laplacian is *positive semidefinite*.

Being real and symmetric, the Laplacian admits an eigendecomposition

$$\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top, \quad (3.12)$$

where the columns of $\mathbf{U} \in \mathbb{R}^{n \times n}$ form an orthonormal basis of eigenvectors and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ collects the corresponding eigenvalues. Symmetry guarantees that these eigenvalues are real, and positive semidefiniteness that they are nonnegative; ordered, they satisfy $0 = \lambda_1 \leq \dots \leq \lambda_n$. The smallest eigenvalue is always zero, with the constant vector as an eigenvector, since the rows of $\mathbf{L}$ sum to zero. More can be said: the multiplicity of the eigenvalue 0 equals the number of connected components of the graph, linking the spectral description back to the connectivity of Section 3.3.

For many purposes the Laplacian is rescaled by the node degrees. The *symmetric normalized Laplacian* is

$$\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}, \quad (3.13)$$

⁶ Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8

where $\mathbf{I}$ is the identity matrix and $\mathbf{D}^{-1/2}$ is the diagonal matrix of inverse square-root degrees. It is again symmetric and positive semidefinite, so it shares the spectral structure just described, while its eigenvalues are confined to the interval $[0, 2]$ ⁷. This normalized form is the variant used in the spectral developments of later chapters. Together, these properties make the Laplacian a natural bridge between the structure of a graph and the linear algebra used to operate on it.

For a node of degree zero the corresponding entry of $\mathbf{D}^{-1/2}$ is taken to be zero by convention.

⁷ Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8

3.5 Notation Summary

The notation introduced in this chapter is collected in Table 3.1 for reference. It is used consistently throughout the remainder of this work.

Symbol	Meaning	Reference
$\mathcal{G} = (V, E)$	Graph with node set V and edge set E	(3.1)
n, m	Number of nodes and edges	§3.1
i, j	Nodes; i central, j a neighbor	§3.1
$\mathcal{N}(i)$	Neighborhood of node i	(3.2)
$\deg(i)$	Degree of node i	(3.3)
$d(i, j)$	Geodesic distance between i and j	(3.6)
π	Isomorphism bijection between node sets	(3.7)
$\mathbf{x}_i, d$	Node feature vector and its dimension	(3.4)
$\mathbf{X}$	Node feature matrix	(3.4)
$\mathbf{e}_{i,j}, c$	Edge feature vector and its dimension	§3.1
w_{ij}	Scalar edge weight	§3.2
$\mathbf{A}$	Adjacency matrix	(3.5)
$(\mathbf{A}^k)_{ij}$	Walks of length k between i and j	(3.8)
$\mathbf{D}$	Degree matrix	(3.9)
$\mathbf{L}$	Graph Laplacian	(3.10)
$\mathbf{L}_{\text{sym}}$	Symmetric normalized Laplacian	(3.13)
$\mathbf{f}$	Graph signal, one real value per node	(3.11)
$\mathbf{U}, \mathbf{\Lambda}$	Eigenvectors and eigenvalues of $\mathbf{L}$	(3.12)
$\lambda_1, \dots, \lambda_n$	Laplacian eigenvalues, in increasing order	(3.12)

Table 3.1: Notation introduced in this chapter, with the equation or section in which each symbol is defined.

4 Neural Networks and Their Limitations on Relational Data

Contents

4.1	The Multilayer Perceptron	35
4.2	Training by Gradient Descent	36
4.3	Why the Multilayer Perceptron Fails on Graphs	38

The preceding chapters established what graphs are and the tasks defined on them. Before constructing a model that operates on graphs, it is necessary to examine the standard tool of deep learning, the multilayer perceptron. This chapter first assembles the multilayer perceptron and the procedure by which it is trained, so that it stands as a complete and well-defined object. It then attempts to apply this model to a graph and shows that the attempt fails. The reasons for that failure are not incidental: they identify precisely the properties a model must possess to operate on graphs and motivate the developments presented in the following chapters.

4.1 The Multilayer Perceptron

The *multilayer perceptron* (MLP) is the elementary feedforward neural network. It maps a fixed-size input vector to an output vector through a sequence of layers, each applying a linear transformation followed by a nonlinear function ¹.

A single layer takes a vector and produces another vector. Writing $\mathbf{h}^{(l-1)}$ for the input to layer l and $\mathbf{h}^{(l)}$ for its output, the layer computes

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad (4.1)$$

where $\mathbf{W}^{(l)}$ is a learnable weight matrix, $\mathbf{b}^{(l)}$ a learnable bias vector, and σ the *activation function*, a fixed nonlinear function applied elementwise. The index $l = 1, \dots, L$ runs over the L layers of the network. The weight matrix and bias vector of each layer are the *parameters* of the

¹ Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

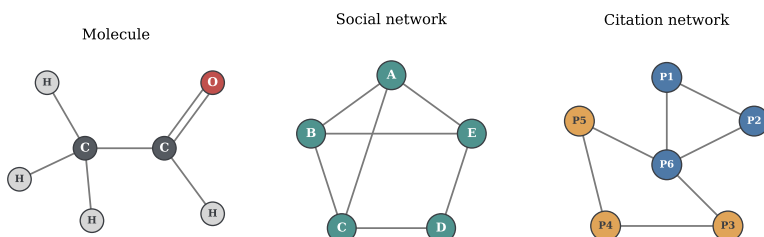
model, collectively denoted by θ , and are the quantities adjusted during training.

The activation function σ is what gives the network its expressive power. Without it, the composition of linear layers would itself be a single linear transformation, no matter how many layers were stacked. A nonlinear σ — for instance the rectified linear unit, $\sigma(x) = \max(0, x)$ — allows the network to represent nonlinear relationships between input and output ².

A multilayer perceptron is the composition of several such layers. Taking the input vector $\mathbf{x}$ as the layer-zero representation, $\mathbf{h}^{(0)} = \mathbf{x}$, the network applies the layers in sequence,

$$f(\mathbf{x}) = f^{(L)} \circ \dots \circ f^{(1)}(\mathbf{x}), \quad (4.2)$$

where each $f^{(l)}$ denotes the transformation of layer l defined in (4.1), and the output of the final layer is the prediction of the network. The layers between the input and the output are the *hidden layers*, and the number of layers L is the *depth* of the network. The map f takes a fixed-size vector $\mathbf{x} \in \mathbb{R}^d$ and returns an output of fixed size, determined by the dimensions of the weight matrices (Figure 4.1).



² Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

Figure 4.1: A multilayer perceptron mapping a fixed-size input vector $\mathbf{x} \in \mathbb{R}^d$ through two hidden layers to an output. Each layer applies a learnable linear transformation followed by an elementwise nonlinearity. The dimension of the input is fixed in advance by the architecture: every input the network processes must be a vector of exactly d components.

This last property is essential to what follows. The architecture fixes the size of the input in advance: the product $\mathbf{W}^{(1)}\mathbf{x}$ is defined only if the input vector $\mathbf{x}$ has the required dimension, and the network assumes that its components are arranged in a definite order. A multilayer perceptron is a function on $\mathbb{R}^d$ for a fixed d , and on nothing else.

4.2 Training by Gradient Descent

The parameters θ are not set by hand but learned from data, by adjusting them so that the network's predictions match ground-truth targets. This requires a measure of how wrong the predictions are and a procedure for reducing it.

The measure is a *loss function*. Given a dataset of n examples with inputs $\mathbf{x}_i$ and targets y_i , the loss aggregates the discrepancy between

each prediction $f(\mathbf{x}_i)$ and its target into a single scalar,

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i), \quad (4.3)$$

where ℓ is a per-example loss, such as the squared error for regression or the cross-entropy for classification. The loss is a function of the parameters θ , since the prediction $f(\mathbf{x}_i)$ depends on them; training is the search for the θ that minimizes $\mathcal{L}(\theta)$.

That search is carried out by *gradient descent*. The gradient $\nabla_{\theta} \mathcal{L}$ points in the direction of steepest increase of the loss; moving the parameters in the opposite direction decreases it. Each step updates the parameters by

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta), \quad (4.4)$$

where the scalar $\eta > 0$ is the *learning rate*, controlling the size of each step. The gradient itself is computed by *backpropagation*, the application of the chain rule across the layers of the network³; it yields the derivative of the loss with respect to every parameter, and is the algorithm that makes training deep networks practical.

³ Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

Computing the gradient in (4.4) over the full dataset is costly when n is large, since it requires a pass over every example for a single update. *Stochastic gradient descent* replaces the full-dataset gradient with an estimate computed on a small random subset of the data, a *minibatch* $\mathcal{B}$. The method was originally defined for a single example per step — the source of the name, as each step follows a noisy, *stochastic* estimate of the gradient rather than the exact one — but is generally applied over a minibatch, which averages the per-example gradients,

$$\nabla_{\theta} \mathcal{L}(\theta) \approx \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta} \ell(f(\mathbf{x}_i), y_i). \quad (4.5)$$

Under uniform random sampling of examples, the minibatch gradient is an unbiased estimate of the full gradient, and each update is far cheaper, so many more updates are performed for the same computational cost. Stochastic gradient descent is the basis of essentially all modern neural network training.

The optimizer used throughout the implementations of this work is *Adam*⁴, a method in the stochastic gradient descent family that adapts the effective step size for each parameter individually, using running estimates of the gradient. Its internal mechanics are not required for the developments that follow, and the reader is referred to the original work for them. What matters is that the multilayer perceptron, with its parameters, loss, and training procedure, now stands as a complete and trainable object. The remainder of the chapter asks whether that object can be applied to a graph.

⁴ Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. <https://arxiv.org/abs/1412.6980>

4.3 Why the Multilayer Perceptron Fails on Graphs

The multilayer perceptron assembled above is a function on a fixed-size vector. A graph is not a vector. To apply the perceptron to a graph, the graph must first be converted into a vector of fixed size, and the difficulty is that no such conversion preserves what the graph encodes. This section attempts the conversion directly and examines what goes wrong; the failures that surface are the substance of the chapter.

Consider a node-level task: predicting a property of each node of a graph, as in the node classification setting of Chapter 2. There are two natural ways to present such a problem to a multilayer perceptron, and it is instructive that both fail, for different reasons.

The first approach attempts to encode the whole graph as a single input vector and feed it to the network. The structure of a graph on n nodes is carried by its adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, and its node attributes by the feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$. To form a single input vector, these matrices are flattened — their entries read out in order and laid end to end — and concatenated. This at once runs into two obstructions.

The first obstruction concerns size. The flattened adjacency matrix has n^2 entries and the flattened feature matrix has $n \times d$; the length of the resulting input vector therefore grows with the number of nodes and differs from one graph to another. But a multilayer perceptron, by (4.1), admits inputs of one fixed dimension only. A network built for a graph of one size cannot accept a graph of another. The same obstruction appears within a single graph. A finer attempt would process each node together with its neighbors, rather than flattening the whole graph at once; but nodes differ in how many neighbors they have, so this neighborhood-based input again has no fixed size (Figure 4.2). This is the problem of *variable size*: neither across graphs nor across the nodes of one graph is the number of entities fixed, whereas the multilayer perceptron demands a fixed-dimensional input.

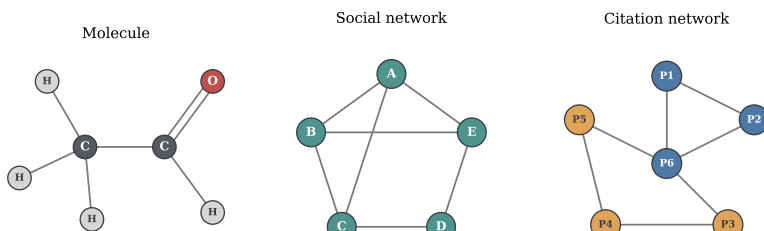
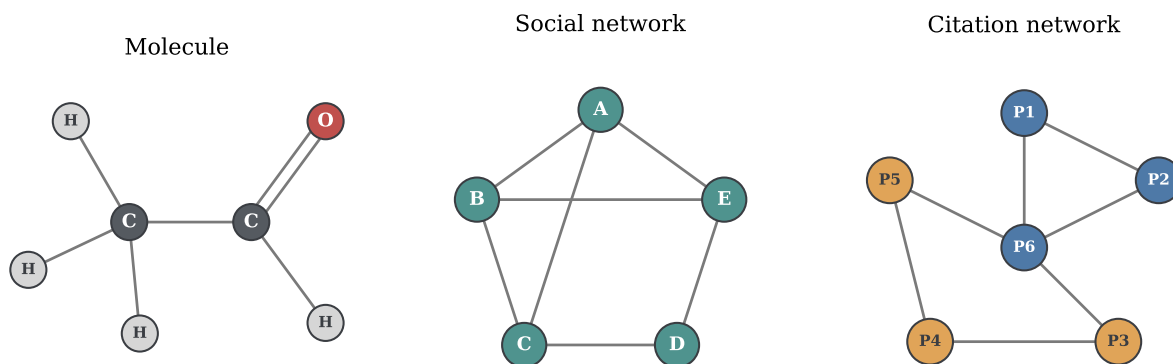


Figure 4.2: Two nodes of the same graph with neighborhoods of different sizes: the node on the left has three neighbors, the node on the right has six. A model that processes a node together with its neighbors must accept inputs of varying size, which a multilayer perceptron cannot.

The second obstruction concerns order. As established in Chapter 3, the nodes of a graph carry no intrinsic ordering; an indexing must be



chosen to write down the adjacency matrix, and any of the $n!$ possible indexings describes the same graph. The two adjacency matrices of a graph under two different node orderings are different matrices, and flattening them yields two different vectors. A multilayer perceptron would receive these as two distinct inputs and could process them differently, even though they represent the same graph (Figure 4.3). The model would thus depend on an arbitrary choice that carries no information — the labeling of the nodes — and would have no means of recognizing that the choice is arbitrary. This is the problem of *node ordering*: the flattened representation imposes an order that the graph does not have, and the multilayer perceptron has no way to disregard it.

Faced with these obstructions, one might abandon the adjacency matrix altogether and take the second approach: present each node to the network on its own, using only its feature vector $\mathbf{x}_i$, and predict the node's property from that. This input has a fixed size d , and the ordering of the nodes no longer enters, since each node is treated separately. Both previous obstructions vanish — but at a cost that defeats the purpose. With the adjacency matrix discarded, the structure of the graph never enters the computation at all. The network processes each node as an isolated example. Yet the tasks of Chapter 2 are relational by nature: in a citation network, a paper's topic is informed by the topics of the papers it cites, and a model that sees a node in isolation discards exactly this evidence. This is the problem of *missing relational structure*: the only input that fits a multilayer perceptron is one from which the relations have been removed.

The two approaches exhaust the natural options, and they fail in opposite ways. Including the adjacency matrix preserves the structure but breaks on variable size and node ordering; excluding it restores a fixed, order-free input but discards the structure entirely. There is

Figure 4.3: A single graph drawn twice with its nodes indexed in two different orders. Each ordering produces a different adjacency matrix, and flattening each matrix produces a different input vector. A multilayer perceptron receives the two vectors as unrelated inputs, although they encode the same graph. Neither ordering is privileged: the graph is the same object in both.

no middle ground: the multilayer perceptron cannot be given a graph without either confronting a representation it cannot accept or being handed one from which the graph has been erased.

These failures reveal that the multilayer perceptron carries the wrong assumptions for relational data. Its *inductive bias* is that examples are independent and of fixed size, an assumption appropriate to vectors but false for graphs, whose entities are connected and whose representation admits no canonical order. What the analysis exposes is the assumptions a graph model must instead possess: it must accept inputs of varying size, it must be *invariant* to the ordering of the nodes, so that relabeling leaves its predictions unchanged, and it must let the structure of the graph participate in the computation. These three requirements, arrived at here by watching the multilayer perceptron fail, are the foundation on which the following chapters build a neural network suited to graphs.

5 Inductive Bias, Invariance, and Equivariance

Contents

5.1	Inductive Bias in Neural Architectures	41
5.2	The Relational Inductive Bias	42
5.3	Invariance and Equivariance	43
5.4	Permutation Invariance and Equivariance on Graphs	44

The preceding analysis in Section 4.3 introduced two concepts in passing — the *inductive bias* of a model and the requirement that a graph model be *invariant* to node ordering — without developing either. This chapter develops both. It first makes precise what an inductive bias is and why successful generalization requires one, reading the inductive biases of successful architectures as evidence that each encodes the structure of its own domain. It then identifies the inductive bias appropriate to graphs and gives the two symmetry conditions that express it formally.

5.1 Inductive Bias in Neural Architectures

A model trained on a finite set of examples must, to be useful, make predictions on examples it has never seen. The training data alone cannot determine those predictions: infinitely many functions are consistent with any finite sample. A model generalizes only because it brings assumptions of its own about which functions are plausible before any data is observed. The set of such assumptions is the *inductive bias* of the model ¹. Generalization is therefore not possible without an inductive bias, and succeeds when that bias matches the structure of the problem.

Deep learning architectures differ primarily in the inductive biases their structure encodes. Three of them recur throughout this work: the multilayer perceptron (MLP), the convolutional neural network (CNN),

¹ Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

and the recurrent neural network (RNN). From this point onward, these architectures are referred to exclusively by their acronyms, both in this chapter and throughout the remainder of the document.

The three differ precisely in the assumptions their structure imposes. The MLP, whose layers connect every input to every output, assumes that any input coordinate may interact with any other, that no particular structure among the coordinates is presumed, and that examples are treated independently. This is the appropriate bias when the input is an unstructured vector — such as tabular data — but the wrong bias for data with an underlying structure.

The CNN and the RNN illustrate how a well-matched bias is built into structure. The CNN, designed for data arranged on a regular grid such as the pixels of an image, applies the same local filter at every position. Two assumptions are encoded in this design. The first is *locality*: a pixel is informative about its immediate neighbors, so the filter reads only a small spatial window rather than the whole image. The second is that a pattern is meaningful regardless of where in the image it appears, so the same filter is reused at every position; a feature shifted across the grid produces the same feature shifted in the output. These assumptions make the CNN well suited to natural images. The RNN, designed for sequences, encodes a different bias: it processes elements in order, applying the same transition at every step. This assumes that the data has a meaningful sequential order and that the rule relating one element to the next remains the same throughout the sequence. These assumptions are appropriate for text and time series.

The same principle appears in all three. Each architecture succeeds when its inductive bias matches the structure of the data, and the match is built into how the architecture is wired rather than learned from data. The MLP assumes no particular structure; the CNN assumes a grid with local, position-independent patterns; the RNN assumes an ordered sequence with a fixed step-to-step rule. The question this chapter must answer follows immediately: what is the structure of a graph, and what inductive bias matches that structure.

5.2 The Relational Inductive Bias

The defining structure of a graph is relational. An entity is not described in isolation but through its connections to other entities: in a citation network the subject of a paper is reflected in the papers it cites, and in a molecule the role of an atom is determined by the atoms bonded to it. The inductive bias appropriate to graphs is therefore the assumption that a node is characterized by its neighborhood — that the information relevant to a node resides in the nodes adjacent to it and in the structure connecting them. This assumption is the *relational inductive bias* ².

² Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261 [cs.LG], 2018. URL <https://arxiv.org/abs/1806.01261>

The relational inductive bias is the graph counterpart of the locality that the CNN encodes, and the comparison is exact in one respect and instructive in its difference. Both assume that an entity is informed by what lies near it rather than by everything at once, and both therefore read from a local region rather than the whole input. The difference lies in what counts as local. On a grid, locality is geometric: a pixel's neighbors are the pixels at fixed offsets around it, the same offsets everywhere, so the notion of a local window is defined once and applied uniformly. On a graph there are no fixed offsets and no uniform window. A node's neighborhood is whatever the edges make it, varying in size from one node to the next. The relational inductive bias retains the principle of locality while discarding the regular geometry that made it simple to express on a grid; locality on a graph is defined by adjacency, not by position.

Adjacency, however, fixes only which nodes are neighbors, not the order in which they are listed. A neighborhood is a set, and a set has no first element. The relational inductive bias says where a node's information comes from; it does not, and must not, depend on the order in which that information is enumerated, because the graph supplies no such order. Making this requirement precise calls for the concepts of the next two sections.

5.3 Invariance and Equivariance

The requirement that a model not depend on an arbitrary choice in its input is made precise by the notions of *invariance* and *equivariance*. Both describe how a function responds to transformations of its input, and they are defined for any function and any transformation before being specialized to graphs.

Let f be a function and let g denote a transformation applied to its input. The function f is *invariant* to g when transforming the input leaves the output unchanged:

$$f(g \cdot \mathbf{x}) = f(\mathbf{x}). \quad (5.1)$$

The transformation is applied to the input, and the output does not move. The function f is *equivariant* to g when transforming the input transforms the output in the same way:

$$f(g \cdot \mathbf{x}) = g \cdot f(\mathbf{x}). \quad (5.2)$$

The transformation is applied to the input, and here the output moves in step with the input.

The distinction is between discarding a transformation and preserving it. Invariance identifies transformed and untransformed

inputs by mapping them to the same output, so the transformation is discarded by f . Equivariance preserves the effect of the transformation: it survives the application of f and acts on the result, so applying f and then transforming gives the same answer as transforming and then applying f . Which of the two is required is not a matter of preference but of the task: when the quantity to be predicted is itself unaffected by the transformation, f must be invariant; when the quantity to be predicted is attached to parts of the input that the transformation moves, f must be equivariant, so that the prediction moves with the part it describes.

5.4 *Permutation Invariance and Equivariance on Graphs*

The transformation that matters for graphs is the reordering of nodes. As established in Chapter 3, the nodes of a graph carry no intrinsic order; an indexing must be chosen to write the adjacency matrix $\mathbf{A}$ and the feature matrix $\mathbf{X}$, and any of the $n!$ possible indexings describes the same graph. A reordering of the nodes is represented by a *permutation matrix* $\mathbf{P} \in \{0, 1\}^{n \times n}$, a square matrix with exactly one entry equal to 1 in each row and each column and zeros elsewhere. An entry $P_{ij} = 1$ indicates that the node originally at index j is mapped to the new index i . Multiplying by $\mathbf{P}$ rearranges the rows of a matrix according to this relabeling. A permutation matrix is orthogonal,

$$\mathbf{P}\mathbf{P}^\top = \mathbf{I}, \quad (5.3)$$

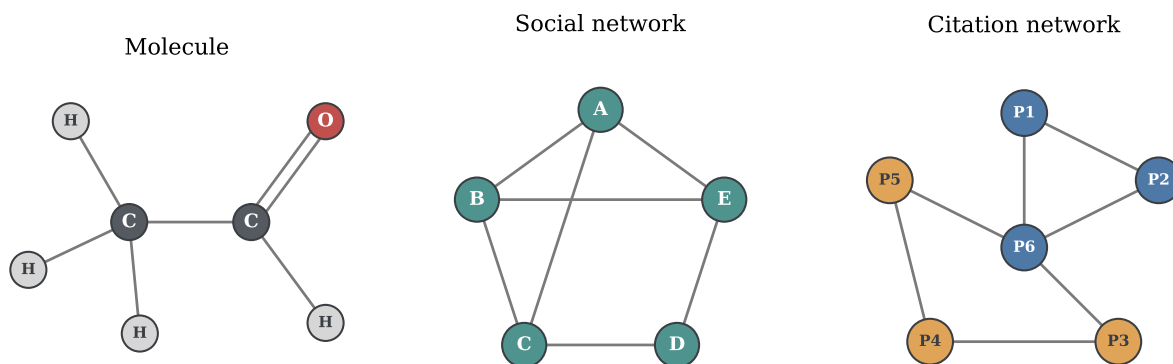
which states that the reordering is invertible and that its inverse is its transpose: the rearrangement can always be undone.

Applying the reordering $\mathbf{P}$ to a graph acts on both of its matrices. The feature matrix becomes $\mathbf{P}\mathbf{X}$, which moves each node's feature vector to the row assigned by the reordering. The adjacency matrix becomes $\mathbf{P}\mathbf{A}\mathbf{P}^\top$, with $\mathbf{P}$ reordering its rows and $\mathbf{P}^\top$ its columns, so that the entry recording an edge between two nodes follows them to their new positions. The pair $(\mathbf{P}\mathbf{A}\mathbf{P}^\top, \mathbf{P}\mathbf{X})$ is the same graph as $(\mathbf{A}, \mathbf{X})$ under a different indexing; no edge has been added or removed, and no feature has changed, only the node ordering.

A function defined on graphs must respect this fact, and which condition it must satisfy depends on the level of the task. For a task whose output describes the graph as a whole — a single prediction for the entire graph, as in graph classification — the output must not depend on the indexing at all. Such a function f must be *permutation invariant*:

$$f(\mathbf{P}\mathbf{A}\mathbf{P}^\top, \mathbf{P}\mathbf{X}) = f(\mathbf{A}, \mathbf{X}). \quad (5.4)$$

Reordering the nodes leaves the graph-level prediction unchanged. For a task whose output is attached to the individual nodes — one



prediction per node, as in node classification — the output must follow the nodes when they are reordered. In this case, f must be *permutation equivariant*:

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = \mathbf{P}f(\mathbf{A}, \mathbf{X}). \quad (5.5)$$

Reordering the nodes reorders the per-node outputs in exactly the same way: the prediction for a given node is the same prediction, carried to that node's new position ³. Figure 5.1 illustrates the two conditions on a single graph drawn under two orderings.

These two conditions, together with the relational inductive bias of Section 5.2, determine what a layer operating on graphs must be. The relational inductive bias fixes where each node's information comes from. Permutation equivariance fixes how the layer must treat the indexing. This is the design the next chapter develops: the message-passing framework gives these requirements a concrete and general form.

Figure 5.1: A single graph shown under two node orderings related by a permutation $\mathbf{P}$. Left: the original indexing, with a per-node output attached to each node and a single graph-level output computed from the whole graph. Right: the reordered indexing, with the graph-level output unchanged (invariance), while the per-node outputs appear in the permuted positions (equivariance).

³ William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

Bibliography

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261 [cs.LG], 2018. URL <https://arxiv.org/abs/1806.01261>.

Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7.

Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8.

Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5 edition, 2017. ISBN 978-3-662-53622-3.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3.

William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly

accurate protein structure prediction with AlphaFold. *Nature*, 596 (7873):583–589, 2021. DOI: 10.1038/s41586-021-03819-2.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. <https://arxiv.org/abs/1412.6980>.

Miller McPherson, Lynn Smith-Lovin, and James M. Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27 (1):415–444, 2001. DOI: 10.1146/annurev.soc.27.1.415.

Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M. Kriege, Martin Grohe, Matthias Fey, and Karsten Borgwardt. Weisfeiler and Leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023. <http://jmlr.org/papers/v24/22-0240.html>.

Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890.

Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294.